# Semester Project: Flee The Maze

A Semester Project Report

| | |
|---|---|
| **Course:** | CSE142 Object Oriented Programming Techniques |
| **Semester:** | Spring 2026 |
| **Instructor:** | Behraj Khan |
| **Due Date:** | 10th May, 2026 |
| **Author(s):** | Ammar Qureshi — ERP: 33410 |
| | Arham Shah — ERP: 33459 |

Department of Computer Science
Institute of Business Administration, Karachi

May 9, 2026

## Abstract

"Flee The Maze" is a first-person 3D horror game developed in C++ using Raylib. The player is placed inside a randomly generated maze and must find the exit while being hunted by AI-controlled ghosts. The game consists of three progressively larger and more difficult levels.

The problem addressed is the design and implementation of an interactive 3D game using object-oriented and algorithmic principles. The maze is generated procedurally using an iterative Depth First Search (DFS) backtracking algorithm built on a custom generic Stack data structure. Ghosts are stored in a custom generic Queue data structure with operator overloading, and they navigate the maze using the A* pathfinding algorithm with manhattan distance heuristic. The Ghost class inherits from an abstract base class Entity and overrides the CheckCollision() and Update() functions. The player is implemented with a First Person Perspective (FPP), also inheriting from the class Entity, and overriding the same functions.

The result is a fully playable horror game that demonstrates the applications and usage of data structure, algorithms, and OOP principles within a real-time interactive context.


**Keywords:** [Object-Oriented Programming, A* Pathfinding, DFS, C++, Inheritance, Polymorphism, Stack, Queue, Raylib, Game Development, Templates,...]

# Contents

# 1 Introduction

## 1.1 Background

Game Development requires the simultaneous management of rendering, AI, input, audio, and memory, making it an ideal domain for demonstrating OOP principles. "Flee The Maze" addresses procedural maze generation, ghost AI, and real-time game state management, each of which maps naturally to OOP concepts. The project applies core principles taught in CSE142 including inheritance, polymorphism, encapsulation, abstraction, templates, dynamic memory management, data structures, and operator overloading throughout the game loop.

## 1.2 Problem Statement

The problem is to design and implement a real-time 3D horror game in which a player navigates a randomly generated maze while being hunted by AI ghosts, with the maze size and ghost count increasing across three levels. This is challenging because it requires multiple complex systems including procedural maze generation, pathfinding AI, collision detection, and real-time rendering, to work correctly and simultaneously within a single game loop. The input is provided by the player each frame through the keyboard (WASD and SHIFT keys) and mouse, while the output is a rendered 3D scene with responsive movement, ghost behaviour, level progression, and game-state transitions. The constraints include ensuring that the maze is perfectly connected, with exactly one path between any two cells. Moreover, both the player and ghosts must navigate the maze without passing through walls, and all systems must run efficiently enough to maintain real-time performance as the maze grows larger each level.

## 1.3 Objectives

1. Design a class hierarchy that models the problem domain.
2. Implement core OOP concepts: encapsulation, inheritance, polymorphism, and abstraction.
3. Demonstrate correct use of constructors, destructors, and operator overloading.
4. Implement and apply custom generic data structures, including a templated Stack for iterative DFS maze generation and a templated Queue with operator overloading for ghost management.

## 1.4 Scope and Limitations

This project includes a fully playable 3D first-person horror game with three levels, procedurally generated mazes, AI ghosts, collision detection, and audio. It does not include multiplayer functionality, saved game states, or difficulty settings. Moreover, all asset files are assumed to be present in their correct directories at runtime, and the game runs at a fixed resolution of 1280x720.

# 2 OOP Design

## 2.1 Class Hierarchy

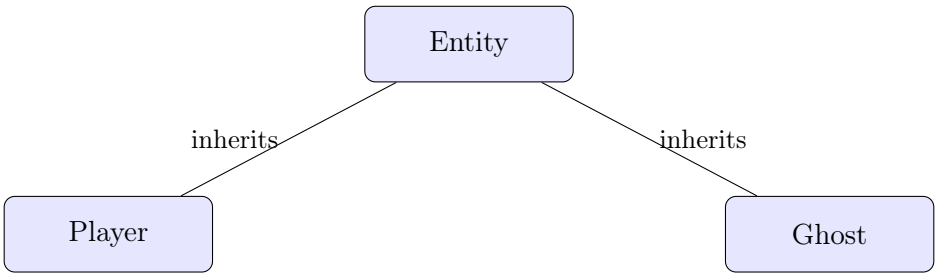


Figure 1: Class hierarchy diagram (is-a relationships)

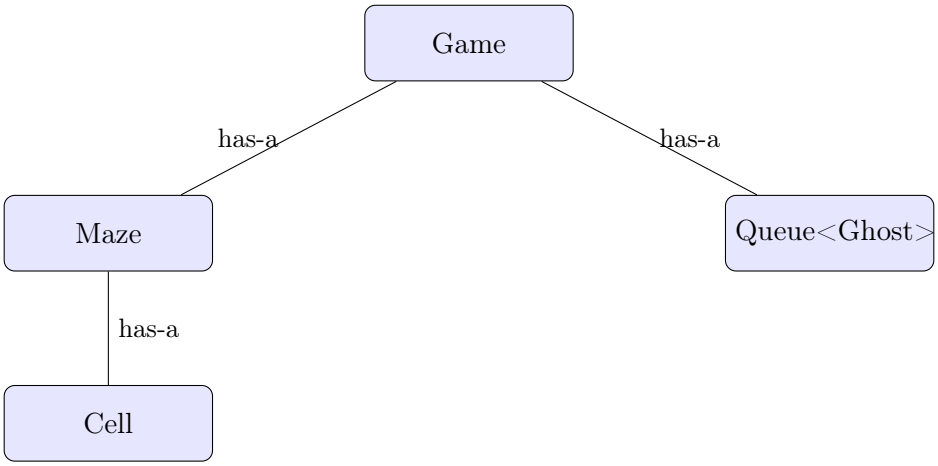


Figure 2: Class hierarchy diagram (has-a relationships)

The game is built around two types of relationships. For inheritance (is-a), the abstract base class **Entity** defines the common interface for all game objects through two pure virtual functions: `Update()` and `checkCollision()`. Both **Player** and **Ghost** inherit from Entity and provide their own concrete implementations of these functions. For composition (has-a), the **Game** class owns a pointer to a **Maze** object which is rebuilt each level, a **Player** object, and a `Queue<Ghost>` which stores all active ghost enemies. The **Maze** class internally owns a 2D vector of **Cell** objects, where each Cell tracks its four walls and visited state. The **Stack** and **Queue** classes are independent generic templated data structures used by Maze and Game respectively.

## 2.2 Class Descriptions

Table 1: Summary of Classes

| Class | Responsibility | Key Members |
|---|---|---|
| `Entity` | Abstract base class for all game objects. Defines common interface through pure virtual functions. | `position, speed, Update(), checkCollision()` |
| `Player` | Inherits from Entity. Handles first-person camera, WASD movement, sprinting, and wall collision. | `camera, Update(), checkCollision()` |
| `Ghost` | Inherits from Entity. Uses A* pathfinding to chase the player. Stores path as vector of cell coords. | `path, target, aStar(), move(), isCaught()` |
| `Game` | Central game manager. Owns all objects, handles game states, rendering, audio, and level progression. | `*maze, player, ghosts, Update(), Draw()` |
| `Maze` | Generates maze using iterative DFS. Owns a 2D vector of Cells. Handles 3D drawing. | `maze[][], generateMaze(), Draw3D()` |
| `Cell` | Represents a single maze cell. Tracks four walls and visited state. | `walls[], visited, hasWall(), removeWall()` |
| `Stack<T>` | Generic templated stack using dynamic array. Used for DFS maze generation. | `push(), pop(), top(), isEmpty()` |
| `Queue<T>` | Generic templated linked-list queue. Used for ghost storage with operator overloading. | `enqueue(), dequeue(), operator+=, operator[]` |

## 2.3 OOP Concepts Applied

- **Encapsulation:** All class member variables are private or protected, accessible only through public getters and setters. For example, `Entity` exposes `getPosition()` and `setPosition()` rather than allowing direct access to `position`. The `Maze` class encapsulates its internal `vector<vector<Cell>>` and exposes only necessary operations like `getCell()`, `isValid()`, and `worldToCell()`.
- **Inheritance:** `Player` and `Ghost` both inherit from the abstract base class `Entity`, which defines the shared interface through pure virtual functions `Update()` and `checkCollision()`. This allows both classes to share common attributes like `position` and `speed` while pro-

viding their own specialized behaviour.

- **Polymorphism:** The pure virtual functions `Update()` and `checkCollision()` in `Entity` are overridden by both `Player` and `Ghost`. This allows the `Game` class to treat both as `Entity` objects through a common interface while each executes its own specific logic at runtime.
- **Operator Overloading:** The `Queue` class overloads several operators: `+=` to merge two ghost queues during level transitions, `[]` for index-based ghost access, `==` and `!=` to compare queue sizes, and `>` to detect hard mode when ghost count exceeds a threshold.
- **Templates:** Both `Stack` and `Queue` are implemented as generic templated classes using `template<typename T>`. `Stack<std::pair<int,int>>` is used for maze generation and `Queue<Ghost>` is used for ghost storage, demonstrating reusable type-independent data structures.

# 3 Implementation

## 3.1 Development Environment

- **Language:** C++17
- **Compiler:** g++ (MinGW-w64)
- **IDE:** Visual Studio Code
- **Build system:** Makefile
- **Platform:** Windows 11
- **Graphics Library:** Raylib 5.5

## 3.2 Base Class (Entity.hpp / .cpp)

```
class Entity {
    protected:
        Vector3 position;
        float speed;
    public:
        Entity(Vector3 spawn = {0, 0, 0}) : position(spawn), speed
            (0.4f) {}
        virtual void Update(const Maze* maze) = 0;
        Vector3 getPosition() const { return position; }
        void setPosition(Vector3 pos) { position = pos; }
        virtual bool checkCollision(const Maze& maze, Vector3 pos)
            const = 0;
        virtual ~Entity() {}
};
```

Listing 1: Base class definition and implementation

## 3.3 Derived Classes (Player.hpp / Ghost.hpp)

```
class Player : public Entity {
    Camera3D camera; //fpp mode
    float shift_speed; //booster
    float yaw, pitch; //for rotation
    Vector3 forward;
```

```
    Vector3 right;
    public:
        Player(Vector3 spawn);
        Player();
        void Update(const Maze* maze) override;
        Camera3D getCamera() const;
        bool checkCollision(const Maze& maze, Vector3 pos) const
           override;
};
class Ghost : public Entity {
    std::vector<std::pair<int, int>> path; //cell coords
    std::pair<int, int> target; //current player cell
    Vector3 targetPos; //current player pos
    float repathTime, repathDelay;
    std::vector<std::pair<int, int>> aStar(
        const Maze& maze,
        std::pair<int, int> start,
        std::pair<int, int> end
    ); //pathfinding
    float manhattan(std::pair<int, int> a, std::pair<int, int> b)
       const;
    void move(const Maze* maze, Vector3 pp);
    public:
        Ghost(Vector3 spawn);
        Ghost(); //position can be set through inherited setPos func
        void setTarget(std::pair<int, int> tar, Vector3 pos);
        void Update(const Maze* maze) override;
        bool checkCollision(const Maze& maze, Vector3 pos) const
           override;
        bool isCaught(Vector3 tarPos) const;
        void Draw(Texture2D mod, Camera3D cam) const;
        void setRepathDelay(float rd);
};
```

Listing 2: Derived class with overridden virtual functions

## 3.4 Demonstrating Polymorphism

In this project, polymorphism is demonstrated through the abstract base class `Entity`, which declares `Update()` and `checkCollision()` as pure virtual functions. Both `Player` and `Ghost` inherit from `Entity` and provide their own concrete implementations. The `Game` class calls `Update()` on both through their respective objects, and each executes its own logic at runtime.

```
void Player::Update(const Maze* maze) {
    float sensitivity = 0.005f;
    Vector2 md = GetMouseDelta();
    yaw -= md.x * sensitivity;
    pitch -= md.y * sensitivity;
    if (pitch>1.5f) pitch = 1.5f;
    if (pitch<-1.5f) pitch = -1.5f;
    Vector3 direc;
    direc.x = cosf(pitch) * sinf(yaw);
    direc.y = sinf(pitch);
    direc.z = cosf(pitch) * cosf(yaw);
    camera.target = {
        camera.position.x + direc.x,
        camera.position.y + direc.y,
        camera.position.z + direc.z
```

```
16      };
17      forward.x = direc.x;
18      forward.y = 0.0f;
19      forward.z = direc.z;
20      float len = sqrtf(forward.x*forward.x + forward.y*forward.y +
            forward.z*forward.z);
21      if (len!=0) {
22          forward.x /= len;
23          forward.y /= len;
24          forward.z /= len;
25      }
26      right.x = forward.y * camera.up.z - forward.z * camera.up.y;
27      right.y = forward.z * camera.up.x - forward.x * camera.up.z;
28      right.z = forward.x * camera.up.y - forward.y * camera.up.x;
29      float len2 = sqrtf(right.x*right.x + right.y*right.y + right.z*
            right.z);
30      if (len2!=0) {
31          right.x /= len2;
32          right.y /= len2;
33          right.z /= len2;
34      }
35      speed = 0.03f;
36      if (IsKeyDown(KEY_LEFT_SHIFT) || IsKeyDown(KEY_RIGHT_SHIFT))
            speed *= shift_speed;
37      if (IsKeyDown(KEY_W)) {
38          Vector3 pos = camera.position;
39          pos.x += forward.x * speed;
40          pos.z += forward.z * speed;
41          if (!checkCollision(*maze, pos)) {
42              camera.position = pos;
43              position = pos;
44          }
45      }
46      if (IsKeyDown(KEY_S)) {
47          Vector3 pos = camera.position;
48          pos.x -= forward.x * speed;
49          pos.z -= forward.z * speed;
50          if (!checkCollision(*maze, pos)) {
51              camera.position = pos;
52              position = pos;
53          }
54      }
55      if (IsKeyDown(KEY_D)) {
56          Vector3 pos = camera.position;
57          pos.x += right.x * speed;
58          pos.z += right.z * speed;
59          if (!checkCollision(*maze, pos)) {
60              camera.position = pos;
61              position = pos;
62          }
63      }
64      if (IsKeyDown(KEY_A)) {
65          Vector3 pos = camera.position;
66          pos.x -= right.x * speed;
67          pos.z -= right.z * speed;
68          if (!checkCollision(*maze, pos)) {
69              camera.position = pos;
70              position = pos;
```

```
            }
        }
    }
bool Player::checkCollision(const Maze& maze, Vector3 pos) const {
    float cells = maze.getCellSize(); //cell size
    int col = (int)(pos.x/cells);
    int row = (int)(pos.z/cells);
    if (row<0 || row>=maze.getRow() || col<0 || col>=maze.getColumn()
       ) return true;
    const Cell& cell = maze.getCell(row, col);
    float ox = fmod(pos.x, cells); //offset X
    float oz = fmod(pos.z, cells); //offset Y
    float ps = 0.2f; //player size
    if (cell.hasWall(0) && oz<ps) return true;
    if (cell.hasWall(1) && ox>(cells-ps)) return true;
    if (cell.hasWall(2) && oz>(cells-ps)) return true;
    if (cell.hasWall(3) && ox<ps) return true;
    return false;
}
void Ghost::Update(const Maze* maze) {
    repathTime += GetFrameTime();
    auto curr = maze->worldToCell(position);
    if (path.empty() || repathTime>=repathDelay) {
        path = aStar(*maze, curr, target);
        repathTime = 0.0f;
    }
    move(maze, targetPos);
}
bool Ghost::checkCollision(const Maze& maze, Vector3 pos) const {
    float cs = maze.getCellSize();
    int r = (int)(pos.z / cs);
    int c = (int)(pos.x / cs);
    if (!maze.isValid(r, c)) return true;
    int cr = (int)(position.z / cs);
    int cc = (int)(position.x / cs);
    if (!maze.isValid(cr, cc)) return true;
    int dr = r - cr;
    int dc = c - cc;
    const Cell& ccell = maze.getCell(cr, cc);
    if (dr==1 && dc==0) return ccell.hasWall(2);
    if (dr==-1 && dc==0) return ccell.hasWall(0);
    if (dr==0 && dc==1) return ccell.hasWall(1);
    if (dr==0 && dc==-1) return ccell.hasWall(3);
    return false;
}
```

Listing 3: Pure virtual functions in Entity overridden by Player and Ghost

## 3.5 Data Structures

This project implements two custom generic data structures. The `Stack<T>` is an array-based templated stack with auto-resize, used exclusively for the iterative DFS backtracking algorithm in `Maze::generateMaze()`. The `Queue<T>` is a linked-list based templated queue used to store and manage all active `Ghost` objects in the `Game` class. The Queue implements operator overloading including `+=` for merging ghost queues between levels, `[]` for index-based access, and `>` for hard mode detection, as well as a custom iterator to support range-based for loops.

```cpp
template <typename T>
class Stack {
    T* arr;
    int capacity;
    int size;
    int topIdx;
    void resize(int s) {
        T* temp = new T[s];
        for (int i=0;i<capacity;i++) temp[i] = arr[i];
        delete[] arr;
        capacity = s;
        arr = temp;
    }
    public:
        Stack(int s = 10) {
            capacity = s;
            size = 0;
            arr = new T[capacity];
            topIdx = -1;
        }
        void push(const T& val) {
            if (isFull()) resize(capacity * 2);
            size++;
            arr[++topIdx] = val;
        }
        T pop() {
            if (isEmpty()) throw std::underflow_error("Stack
                underflow");
            size--;
            return arr[topIdx--];
        }
        T& top() const {
            if (isEmpty()) throw std::underflow_error("Stack
                underflow");
            return arr[topIdx];
        }
        int getSize() const { return size; }
        bool isEmpty() const { return size == 0; }
        bool isFull() const { return size == capacity; }
        ~Stack() {
            delete[] arr;
        }
};
```

Listing 4: Stack data structure used for DFS maze generation

```cpp
template <typename T>
class Queue {
    struct node {
        T value;
        node* next;
        node(T val) : value(val), next(nullptr) {}
    };
    node* front;
    node* rear;
    int size;
    public:
        Queue() : size(0) {
```

```
13              front = rear = nullptr;
14          }
15          Queue(const Queue<T>& q) : size(0) {
16              front = rear = nullptr;
17              node* temp = q.front;
18              while (temp) {
19                  enqueue(temp->value);
20                  temp = temp->next;
21              }
22          }
23          Queue<T>& operator = (const Queue<T>& q) {
24              if (this==&q) return *this;
25              clear();
26              node* temp = q.front;
27              while (temp) {
28                  enqueue(temp->value);
29                  temp = temp->next;
30              }
31              return *this;
32          }
33          bool isEmpty() const {
34              return size == 0;
35          }
36          void enqueue(T val) {
37              node* temp = new node{val};
38              if (rear==nullptr) front = rear = temp;
39              else {
40                  rear->next = temp;
41                  rear = temp;
42              }
43              size++;
44          }
45          T dequeue() {
46              if (isEmpty()) throw std::runtime_error("Queue is empty!"
                    );
47              node* temp = front;
48              T val = temp->value;
49              front = front->next;
50              if (front==nullptr) rear = nullptr;
51              delete temp;
52              size--;
53              return val;
54          }
55          T& peek() {
56              if (isEmpty()) throw std::runtime_error("Queue is empty!"
                    );
57              return front->value;
58          }
59          const T& peek() const {
60              if (isEmpty()) throw std::runtime_error("Queue is empty!"
                    );
61              return front->value;
62          }
63          int getSize() const {
64              return size;
65          }
66          void clear() {
67              while (!isEmpty()) dequeue();
```

```
68          }
69
70          T& operator [](int idx) {
71              if (idx<0 || idx>=size) throw std::out_of_range("Index
                    out of bounds!");
72              node* temp = front;
73              for (int i=0;i<idx;i++) temp = temp->next;
74              return temp->value;
75          }
76          const T& operator [](int idx) const {
77              if (idx<0 || idx>=size) throw std::out_of_range("Index
                    out of bounds!");
78              node* temp = front;
79              for (int i=0;i<idx;i++) temp = temp->next;
80              return temp->value;
81          }
82          Queue<T>& operator += (const Queue<T>& q) {
83              node* temp = q.front;
84              while (temp) {
85                  enqueue(temp->value);
86                  temp = temp->next;
87              }
88              return *this;
89          }
90          //comps based on size
91          bool operator == (const Queue<T>& q) const { return size == q
                .size; }
92          bool operator != (const Queue<T>& q) const { return size != q
                .size; }
93          bool operator > (const Queue<T>& q) const { return size > q.
                size; }
94
95          //iterators
96          struct it {
97              node* temp;
98              it(node* n) : temp(n) {}
99              T& operator * () { return temp->value; }
100             T* operator -> () { return &temp->value;}
101             it& operator ++ () {
102                 temp = temp->next;
103                 return *this;
104             }
105             bool operator != (const it& i) const { return temp != i.
                    temp; }
106         };
107         struct const_it {
108             node* temp;
109             const_it(node* n) : temp(n) {}
110             const T& operator * ()  const { return temp->value; }
111             const T* operator -> () const { return &temp->value; }
112             const_it& operator ++ () {
113                 temp = temp->next;
114                 return *this;
115             }
116             bool operator != (const const_it& i) const { return temp
                    != i.temp; }
117         };
118         it begin() { return it(front); }
```

```
        it end()   { return it(nullptr); }
        const_it begin() const { return const_it(front); }
        const_it end() const { return const_it(nullptr); }

        ~Queue() {
            clear();
        }
};
```

Listing 5: Queue data structure used for ghost storage with operator overloading

## 3.6 Additional Key Functions

The following key functions demonstrate important implementation details of this project:

```
std::vector<std::pair<int, int>> Ghost::aStar(
        const Maze& maze,
        std::pair<int, int> start,
        std::pair<int, int> end
) {
    std::map<std::pair<int, int>, bool> inOpen, inClosed;
    std::vector<std::pair<int, int>> open;
    std::map<std::pair<int, int>, std::pair<int, int>> parent;
    std::map<std::pair<int, int>, float> gScore;
    open.push_back(start);
    inOpen[start] = true;
    gScore[start] = 0.0f;
    std::vector<std::pair<int, int>> dir = {
        {1, 0}, {-1, 0},
        {0, 1}, {0, -1}
    };
    while (!open.empty()) {
        int bestIdx = 0;
        float bestF = gScore[open[0]] + manhattan(open[0], end);
        for (size_t i=1; i<open.size();i++) {
            float f = gScore[open[i]] + manhattan(open[i], end);
            if (f<bestF) {
                bestF = f;
                bestIdx = (int)i;
            }
        }
        auto curr = open[bestIdx];
        open.erase(open.begin() + bestIdx);
        if (curr==end) {
            std::vector<std::pair<int, int>> res;
            auto step = end;
            while (parent.count(step)) {
                res.push_back(step);
                step = parent[step];
            }
            std::reverse(res.begin(), res.end());
            return res;
        }
        inClosed[curr] = true;
        for (auto& x : dir) {
            int r = curr.first + x.first;
            int c = curr.second + x.second;
            std::pair<int, int> next = {r, c};
```

```
            if (!maze.isValid(r, c)) continue;
            if (inClosed.count(next)) continue;
            float cs = maze.getCellSize();
            Vector3 np = {
                c * cs + cs/2.0f,
                position.y,
                r * cs + cs/2.0f
            };
            if (checkCollision(maze, np)) continue;
            float newG = gScore[curr] + 1.0f;
            if (!gScore.count(next) || newG<gScore[next]) {
                gScore[next] = newG;
                parent[next] = curr;
                if (!inOpen.count(next)) {
                    open.push_back(next);
                    inOpen[next] = true;
                }
            }
        }
    }
    return {};
}
```

Listing 6: A* pathfinding algorithm in Ghost

```
void Maze::generateMaze() {
    Stack<std::pair<int, int>> s;
    int rs = 0, rc = 0;
    maze[rs][rc].setVisited(true);
    s.push({rs, rc});
    while (!s.isEmpty()) {
        auto [r, c] = s.top();
        std::vector<std::pair<int, int>> neighbors;
        std::vector<int> direc;
        for (int i=0;i<4;i++) {
            int nr = r + dirR[i];
            int nc = c + dirC[i];
            if (isValid(nr, nc) && !maze[nr][nc].isVisited()) {
                neighbors.push_back({nr, nc});
                direc.push_back(i);
            }
        }
        if (!neighbors.empty()) {
            int idx = std::rand() % neighbors.size();
            int nr = neighbors[idx].first;
            int nc = neighbors[idx].second;
            int dir = direc[idx];
            if (dir<0 || dir>3) throw std::out_of_range("Invalid
               direction in maze!");
            maze[r][c].removeWall(dir);
            maze[nr][nc].removeWall(getOppositeWall(dir));
            maze[nr][nc].setVisited(true);
            s.push({nr, nc});
        }
        else s.pop();
    }
    int entry = std::rand() % 4;
    int exit;
```

```
33      do {
34          exit = std::rand() % 4;
35      } while (exit == entry);
36      if (entry==0) {
37          entryRow = 0;
38          entryColumn = std::rand() % columns;
39          Entrance = {entryRow, entryColumn};
40          maze[entryRow][entryColumn].removeWall(0);
41      }
42      else if (entry==1) {
43          entryRow = std::rand() % rows;
44          entryColumn = columns - 1;
45          Entrance = {entryRow, entryColumn};
46          maze[entryRow][entryColumn].removeWall(1);
47      }
48      else if (entry==2) {
49          entryRow = rows - 1;
50          entryColumn = std::rand() % columns;
51          Entrance = {entryRow, entryColumn};
52          maze[entryRow][entryColumn].removeWall(2);
53      }
54      else {
55          entryRow = std::rand() % rows;
56          entryColumn = 0;
57          Entrance = {entryRow, entryColumn};
58          maze[entryRow][entryColumn].removeWall(3);
59      }
60      if (exit==0) {
61          exitRow = 0;
62          exitColumn = std::rand() % columns;
63          Exit = {exitRow, exitColumn};
64          maze[exitRow][exitColumn].removeWall(0);
65      }
66      else if (exit==1) {
67          exitRow = std::rand() % rows;
68          exitColumn = columns - 1;
69          Exit = {exitRow, exitColumn};
70          maze[exitRow][exitColumn].removeWall(1);
71      }
72      else if (exit==2) {
73          exitRow = rows - 1;
74          exitColumn = std::rand() % columns;
75          Exit = {exitRow, exitColumn};
76          maze[exitRow][exitColumn].removeWall(2);
77      }
78      else {
79          exitRow = std::rand() % rows;
80          exitColumn = 0;
81          Exit = {exitRow, exitColumn};
82          maze[exitRow][exitColumn].removeWall(3);
83      }
84  }
```

Listing 7: DFS maze generation using custom Stack

# 4 Testing

## 4.1 Test Cases

Table 2: Test Cases and Results

| TC # | Input / Action | Expected Output | Actual Output | Pass? |
|---|---|---|---|---|
| TC-01 | Game launches | 3D maze renders with player at entrance | As expected | ✓ |
| TC-02 | Player reaches the exit cell | Winner sound plays, loading screen appears | As expected | ✓ |
| TC-03 | Ghost catches the player | Jumpscare sound, red screen, level restarts | As expected | ✓ |
| TC-04 | Player completes level 3 | Gold celebration screen appears, game resets | As expected | ✓ |
| TC-05 | Ghost pathfinds through maze | Ghost navigates only through open passages | As expected | ✓ |
| TC-06 | Player completes a level | Maze size increases by 5, ghost count increases by 2 | As expected | ✓ |

### 4.2 Edge Cases

Table 3: Edge Cases and Handling

| Edge Case | Description | How Handled |
|---|---|---|
| Ghost spawns too close to player | Ghost could spawn directly on top of player at level start | Spawn coordinates regenerated until ghost is at least 3 cells from entrance |
| Ghost and player in same cell | A* returned empty path causing ghost to freeze | Removed early exit from A*, ghost moves directly toward player world position |
| Maze floor disappearing on level up | Grass plane was sized for level 1 only | Floor plane and star positions recalculated each level to match new maze size |
| Ghost pathfinding through walls | Positional collision check failed at cell centres | Replaced with direct `hasWall()` checks in A* neighbour expansion |
| Dequeue on empty Queue | Calling dequeue on empty queue causes undefined behaviour | Queue throws `std::runtime_error` if dequeue or peek called on empty queue |
| Stack capacity exceeded during DFS | On large mazes the DFS stack could exceed initial capacity of 10 | Stack automatically resizes to double capacity when full, preventing any fixed-size limitation |
| Stack underflow on empty stack | Calling `pop()` or `top()` on an empty stack causes undefined behaviour | Stack throws `std::underflow_error` if `pop()` or `top()` is called when stack is empty |

## 5 Results and Discussion

### 5.1 Results

The game successfully implements all core features across three levels. The following table summarises the performance characteristics of the key operations:

Table 4: Performance / Benchmark Results

*Note: Timing values are approximate estimates based on algorithm complexity. Frame rate measured on the development machine.*

| Operation | Input Size | Time (ms) | Notes |
|---|---|---|---|
| Maze Generation (DFS) | 5x5 | <1 | Level 1 |
| Maze Generation (DFS) | 10x10 | <1 | Level 2 |
| Maze Generation (DFS) | 15x15 | <1 | Level 3 |
| A* Pathfinding | 5x5 | <1 | 1 ghost |
| A* Pathfinding | 10x10 | <2 | 3 ghosts |
| A* Pathfinding | 15x15 | <5 | 5 ghosts |
| Frame Rate | All levels | 60 fps | Stable on Intel UHD 620 |

### 5.2 Analysis

**Design Objectives:** All core design objectives were met. The class hierarchy with `Entity` as an abstract base class was successfully implemented with `Player` and `Ghost` as derived classes. Custom generic `Stack` and `Queue` data structures were implemented and actively used in maze generation and ghost management respectively. The game runs across three levels with increasing maze size and ghost count, and all game states including PLAYING, JUMPSCARE, LOADING, and CLEAR function correctly.

**Complexity Analysis:**

- **Maze Generation (DFS):** $O(r \times c)$ where $r$ and $c$ are rows and columns. Every cell is visited exactly once. Stack operations are $O(1)$ on average with auto-resize. Matches theoretical expectation.
- **A* Pathfinding:** $O(n^2 \log n)$ in our implementation; the open list is a `vector` requiring $O(n)$ linear scan per iteration to find the minimum $f$ value, and each `map` operation (for `gScore`, `parent`, `inOpen`, `inClosed`) adds an $O(\log n)$ factor. Theoretical A* with a binary heap priority queue would be $O(n \log n)$.
- **Queue operations:** `enqueue` and `dequeue` are $O(1)$. Index access via `operator[]` is $O(n)$ due to linked list traversal.

**Unexpected Behaviour and Limitations:**

- Ghost movement occasionally froze when the ghost and player occupied the same cell, resolved by removing the early exit from A* and moving directly toward the player's world position.
- The grass floor disappeared on higher levels due to the plane being sized only at initialization, resolved by rebuilding the plane each level.
- Performance degrades slightly on level 3 with 5 ghosts each repathing every 0.8 seconds on a 15x15 maze, due to the $O(n^2)$ A* implementation.

## 6 Challenges and Solutions

### 6.1 Ghost Pathfinding Through Walls

**Problem:** Ghosts were passing through maze walls because the A* collision check tested the centre of neighbouring cells, which never triggered wall detection since cell centres are always far from walls.

**Solution:** Replaced the positional collision check with direct `hasWall()` checks on the current cell, verifying whether a wall exists between the current and neighbouring cell before expanding that node.
**Learning:** Collision detection must be done at the logical level (wall flags) rather than the geometric level (world positions) when working with discrete grid-based mazes.

### 6.2 Ghost Freezing Near Player

**Problem:** When the ghost and player occupied the same cell, A* returned an empty path causing the ghost to completely stop moving.
**Solution:** Removed the early `if (start==end) return` exit from A*, and modified `move()` to move directly toward the player's exact world position when the path is empty.
**Learning:** Pathfinding algorithms that operate on discrete cells need special handling for the final approach, since cell-level resolution is too rough for smooth close-range movement.

## 7 Future Work

Possible extensions and improvements:

- Add a menu screen with Play, Tutorial, and Exit buttons for a more complete user experience.
- Improve lighting using Raylib shaders to create a torch effect, limiting visibility and increasing the horror atmosphere.
- Differentiate the entrance and exit using distinct icons or markers so the player can identify them at a glance.
- Introduce collectible objectives that must be completed before the exit unlocks, adding an exploration objective.
- Implement a lives system so the player has multiple chances before a full game over, rather than restarting the level on every catch.

## 8 Conclusion

"Flee The Maze" successfully demonstrates the practical application of OOP principles in the development of a real-time 3D horror game. The project addressed the challenges of procedural maze generation, ghost AI, and real-time game state management using a well-structured class hierarchy built around an abstract `Entity` base class with `Player` and `Ghost` as derived classes.

The custom generic `Stack` and `Queue` data structures were implemented from scratch and actively used throughout the project; the Stack for iterative DFS maze generation and the Queue for ghost storage with operator overloading. The A* pathfinding algorithm with Manhattan distance heuristic enables ghosts to navigate the maze, and all three levels function correctly with increasing difficulty.

The most valuable learning outcomes were the importance of logical over geometric collision detection in grid-based systems, the challenges of integrating multiple real-time systems within a single game loop, and how OOP principles such as inheritance, polymorphism, encapsulation, and abstraction directly improve code organization and management in complex projects.

## References

List references in IEEE format. Example entries:

[1] *cppreference.com*. [Online]. Available: `https://en.cppreference.com`. [Accessed: May 10, 2026].

[2] Raylib. *Raylib Documentation*. [Online]. Available: `https://www.raylib.com`. [Accessed: May 10, 2026].

[3] A. Qureshi, *Flee The Maze — Source Code*. GitHub. [Online]. Available: `https://github.com/asqtecki/Flee-The-Maze`. [Accessed: May 10, 2026].

[4] *Introduction to Algorithms*, T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, 4th ed. MIT Press, 2022.

# A Complete Source Code

Full source code is available on GitHub at: `https://github.com/asqtecki/Flee-The-Maze`
Few code snippets are pasted below, while others can be found at the given URL of my GitHub repository.

## A.1 main.cpp

```cpp
#include <raylib.h>
#include <iostream>
#include "include/Game.hpp"
#include <cstdlib>
using namespace std;

struct Screen {
    const int width = 1280;
    const int height = 720;
};

int main() {
    srand(time(0));
    Screen s;
    InitWindow(s.width, s.height, "Flee the Maze!");
    DisableCursor();
    SetTargetFPS(60);
    Texture2D wall = LoadTexture("Graphics/wall.jpg");
    Mesh cube = GenMeshCube(1.0f, 1.0f, 1.0f);
    Model wallModel = LoadModelFromMesh(cube);
    wallModel.materials[0].maps[MATERIAL_MAP_DIFFUSE].texture = wall;
    Game g(wallModel);
    while (!WindowShouldClose()) {
        g.Update();
        g.Draw();
    }
    UnloadModel(wallModel);
    UnloadTexture(wall);
    CloseWindow();
}
```

Listing 8: main.cpp — the driver

## A.2 Entity.hpp

```cpp
#pragma once

#include <raylib.h>
#include "Maze.hpp"

class Entity {
    protected:
        Vector3 position;
        float speed;
    public:
        Entity(Vector3 spawn = {0, 0, 0}) : position(spawn), speed
            (0.4f) {}
        virtual void Update(const Maze* maze) = 0;
        Vector3 getPosition() const { return position; }
```

```
14          void setPosition(Vector3 pos) { position = pos; }
15          virtual bool checkCollision(const Maze& maze, Vector3 pos)
                const = 0;
16          virtual ~Entity() {}
17  };
```

Listing 9: Entity.hpp — abstract base class

## A.3 Ghost.hpp

```
1   #pragma once
2
3   #include "Entity.hpp"
4   #include "Maze.hpp"
5   #include <vector>
6   #include <utility>
7
8   class Ghost : public Entity {
9       std::vector<std::pair<int, int>> path; //cell coords
10      std::pair<int, int> target; //current player cell
11      Vector3 targetPos; //current player pos
12      float repathTime, repathDelay;
13      std::vector<std::pair<int, int>> aStar(
14          const Maze& maze,
15          std::pair<int, int> start,
16          std::pair<int, int> end
17      ); //pathfinding
18      float manhattan(std::pair<int, int> a, std::pair<int, int> b)
            const;
19      void move(const Maze* maze, Vector3 pp);
20      public:
21          Ghost(Vector3 spawn);
22          Ghost(); //position can be set through inherited setPos func
23          void setTarget(std::pair<int, int> tar, Vector3 pos);
24          void Update(const Maze* maze) override;
25          bool checkCollision(const Maze& maze, Vector3 pos) const
                override;
26          bool isCaught(Vector3 tarPos) const;
27          void Draw(Texture2D mod, Camera3D cam) const;
28          void setRepathDelay(float rd);
29  };
```

Listing 10: Ghost.hpp — derived class with A* pathfinding

## A.4 Player.hpp

```
1   #pragma once
2   #include <raylib.h>
3   #include "Maze.hpp"
4   #include "Entity.hpp"
5
6   class Player : public Entity {
7       Camera3D camera; //fpp mode
8       float shift_speed; //booster
9       float yaw, pitch; //for rotation
10      Vector3 forward;
11      Vector3 right;
```

```
12      public:
13          Player(Vector3 spawn);
14          Player();
15          void Update(const Maze* maze) override;
16          Camera3D getCamera() const;
17          bool checkCollision(const Maze& maze, Vector3 pos) const
                override;
18  };
```

Listing 11: Player.hpp — Derived class

## A.5 Cell.hpp

```
1  //Represent single grid unit in Maze
2  //Stores position
3  //Tracks wether a unit has been visited or not, for maze generation
4
5  #pragma once
6
7  class Cell {
8      int rows, columns;
9      bool visited;
10     bool walls[4]; //0 = top/north, 1 = right/east, 2 = bottom/south,
           3 = left/west
11     public:
12         Cell();
13         Cell(int r, int c);
14         int getRow() const;
15         int getColumn() const;
16         bool isVisited() const; //checks if unit has been visited
17         void setVisited(bool v);
18         bool hasWall(int idx) const; //checks if a wall exists
19         void removeWall(int idx); //removes a certain wall
20         void setWall(int idx, bool v); //sets wall state
21 };
```

Listing 12: Cell.hpp – Represents a unit within the maze

## A.6 Cell.cpp

```
1  #include "../include/Cell.hpp"
2  #include <stdexcept>
3
4  Cell::Cell() : rows(0), columns(0), visited(false) {
5      for (int i=0;i<4;i++) walls[i] = true;
6  }
7
8  Cell::Cell(int r, int c) : rows(r), columns(c), visited(false) {
9      for (int i=0;i<4;i++) walls[i] = true;
10 }
11
12 int Cell::getRow() const {
13     return rows;
14 }
15
16 int Cell::getColumn() const {
17     return columns;
```

```cpp
}

bool Cell::isVisited() const {
    return visited;
}

void Cell::setVisited(bool v) {
    visited = v;
}

bool Cell::hasWall(int idx) const {
    if (idx>=0 && idx<4) return walls[idx];
    throw std::out_of_range("Index out of bounds!");
}

void Cell::removeWall(int idx) {
    if (idx>=0 && idx<4) {
        walls[idx] = false;
        return;
    }
    throw std::out_of_range("Index out of bounds!");
}

void Cell::setWall(int idx, bool v) {
    if (idx>=0 && idx<4) {
        walls[idx] = v;
        return;
    }
    throw std::out_of_range("Index out of bounds!");
}
```

Listing 13: Cell.cpp — Cell class definition

## A.7 Maze.hpp

```cpp
#pragma once

#include <vector>
#include "Cell.hpp"
#include "Stack.hpp"
#include <utility>
#include <cstdlib>
#include <raylib.h>

class Maze {
    int rows, columns; //height and width respectively
    int entryRow, entryColumn; //entrance coordinates
    int exitRow, exitColumn; //exit coordinates
    float cellSize = 2.0f; //size of each cell
    std::pair<int, int> Entrance;
    std::pair<int, int> Exit;
    std::vector<std::vector<Cell>> maze; //accumulates cells to form
        a maze
    const int dirR[4] = {-1, 0, 1, 0}; //row direction for north,
        east, south, west
    const int dirC[4] = {0, 1, 0, -1}; //column direction for north,
        east, south, west
```

```
20      int getOppositeWall(int dir) const; //returns the opposite wall
            index
21      public:
22          Maze(int r, int c);
23          void generateMaze(); //generates a maze using DFS
24          Cell& getCell(int r, int c); //returns a reference to a cell
                at (r, c)
25          const Cell& getCell(int r, int c) const; //returns a const
                reference to a cell at (r, c)
26          void printMaze() const; //prints the maze to the console
27          void Draw3D(Model& wallM) const; //for raylib drawing
28          int getRow() const;
29          int getColumn() const;
30          float getCellSize() const;
31          std::pair<int, int> getEntrance() const;
32          std::pair<int, int> getExit() const;
33          std::pair<int, int> worldToCell(Vector3 pos) const;
34          bool isValid(int r, int c) const; //checks if a cell is
                within bounds
35          bool isWall(int r, int c) const;
36  };
```

Listing 14: Maze.hpp — Maze class declaration

## A.8 Game.hpp

```
1   #pragma once
2
3   #include <raylib.h>
4   #include "Maze.hpp"
5   #include "Player.hpp"
6   #include "Ghost.hpp"
7   #include "Queue.hpp"
8   #include <vector>
9
10  enum GameState {
11      PLAYING, //1
12      JUMPSCARE, //2
13      LOADING, //3
14      CLEAR, //4
15  };
16
17  class Game {
18      struct Star {
19          Vector3 position;
20          float size, twinkles;
21      };
22      //sounds and music
23      Music permBgSound;
24      Sound exit;
25      Sound jumpscare;
26      bool exitPlayed;
27
28      std::vector<Star> stars;
29      Queue<Ghost> ghosts;
30      int ghostPerLevel;
31      Maze* maze;
32      Player player;
```

```
33      Texture2D grass;
34      Texture2D ghost;
35      GameState state;
36      int mazeSize;
37      int level;
38      float loadingTimer, loadingDuration;
39      Model wallModel;
40      Model grassModel;
41      float jumpscareTimer, jumpscareDuration;
42      bool isAtExit() const;
43      public:
44          Game(Model wall);
45          void Update();
46          void Draw();
47          ~Game();
48  };
```

Listing 15: Game.hpp — Combine everything

# B Installation and Usage

The project is available on GitHub at: `https://github.com/asqtecki/Flee-The-Maze`

## B.1 Prerequisites

- A C++17-compatible compiler (e.g., `g++`)
- `Make` for building the project
- Windows environment with `w64devkit` (recommended)
- Raylib 5.5 installed at `C:/raylib/raylib`

## B.2 Setup and Compilation

### Step 1: Clone the repository

```
1 git clone https://github.com/asqtecki/Flee-The-Maze.git
2 cd Flee-The-Maze
```

### Step 2: Configure compiler (Windows only)

```
1 set PATH=C:\raylib\w64devkit\bin;%PATH%
```

### Step 3: Build the project

```
1 make clean
2 make
```

## B.3 Run the Game

```
1 game.exe
```

*Note: Run `game.exe` from the project root directory to ensure the `Graphics/` and `Music/` folders are found at their correct relative paths.*

## B.4 Controls

- **W, A, S, D**: Move the player
- **Shift**: Sprint
- **Mouse**: Look around (first-person view)
- **ESC**: Exit the game

## B.5 Gameplay Instructions

- The player starts at the maze entrance.
- Navigate through the maze to reach the exit.
- Each level increases the maze size and difficulty.
- Ghosts use pathfinding to track the player.
- Avoid ghosts and reach the exit to progress.